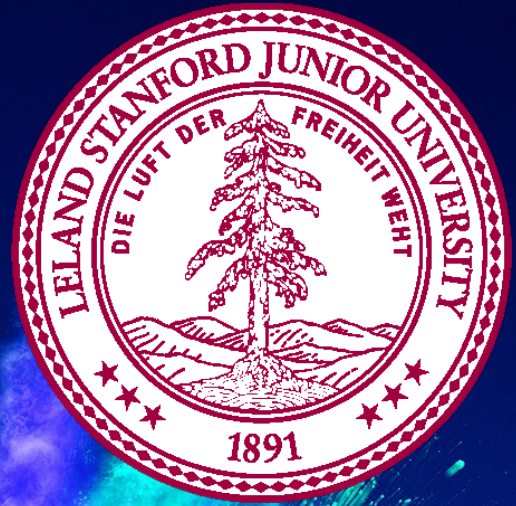




Graphics

Chris Piech and Mehran Sahami
CS106A, Stanford University

Breakout Demo

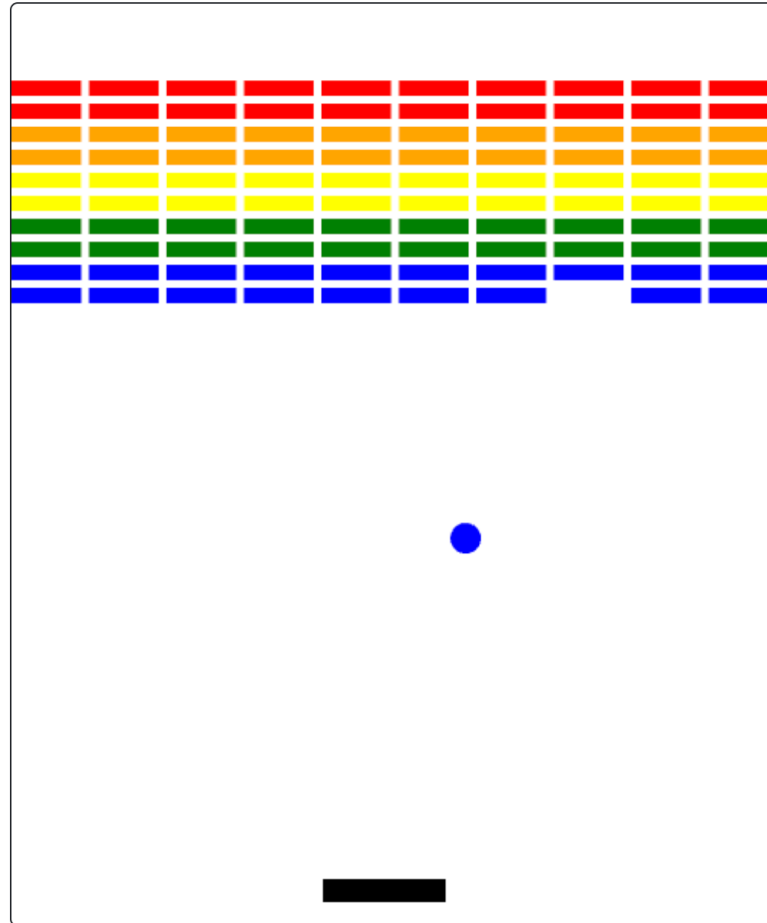


Created by Chris Piech ❤️

Hello! I made this for CS106A!

Editable (Owner)

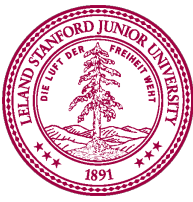
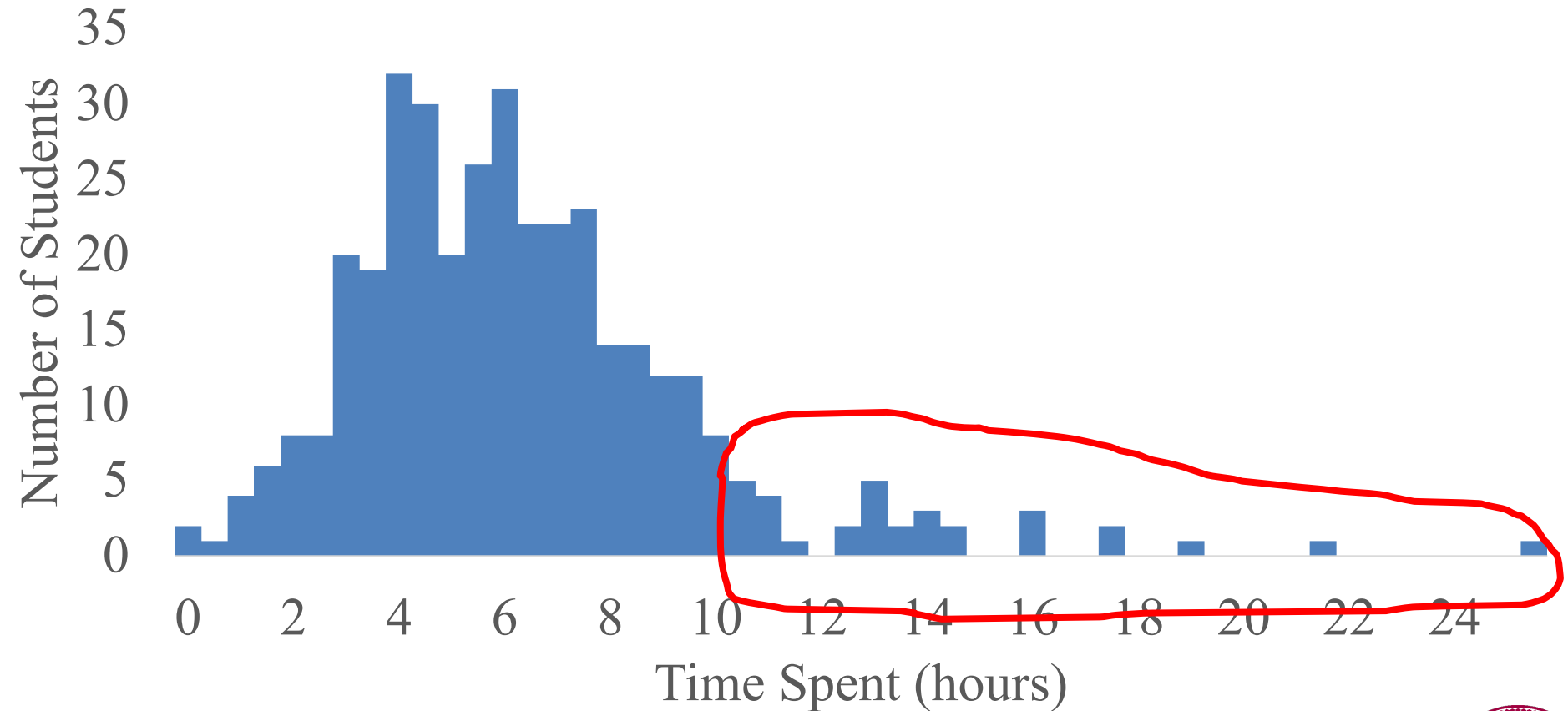
▶ Run



Get a little meta...

Learn by Doing

Assignment 2



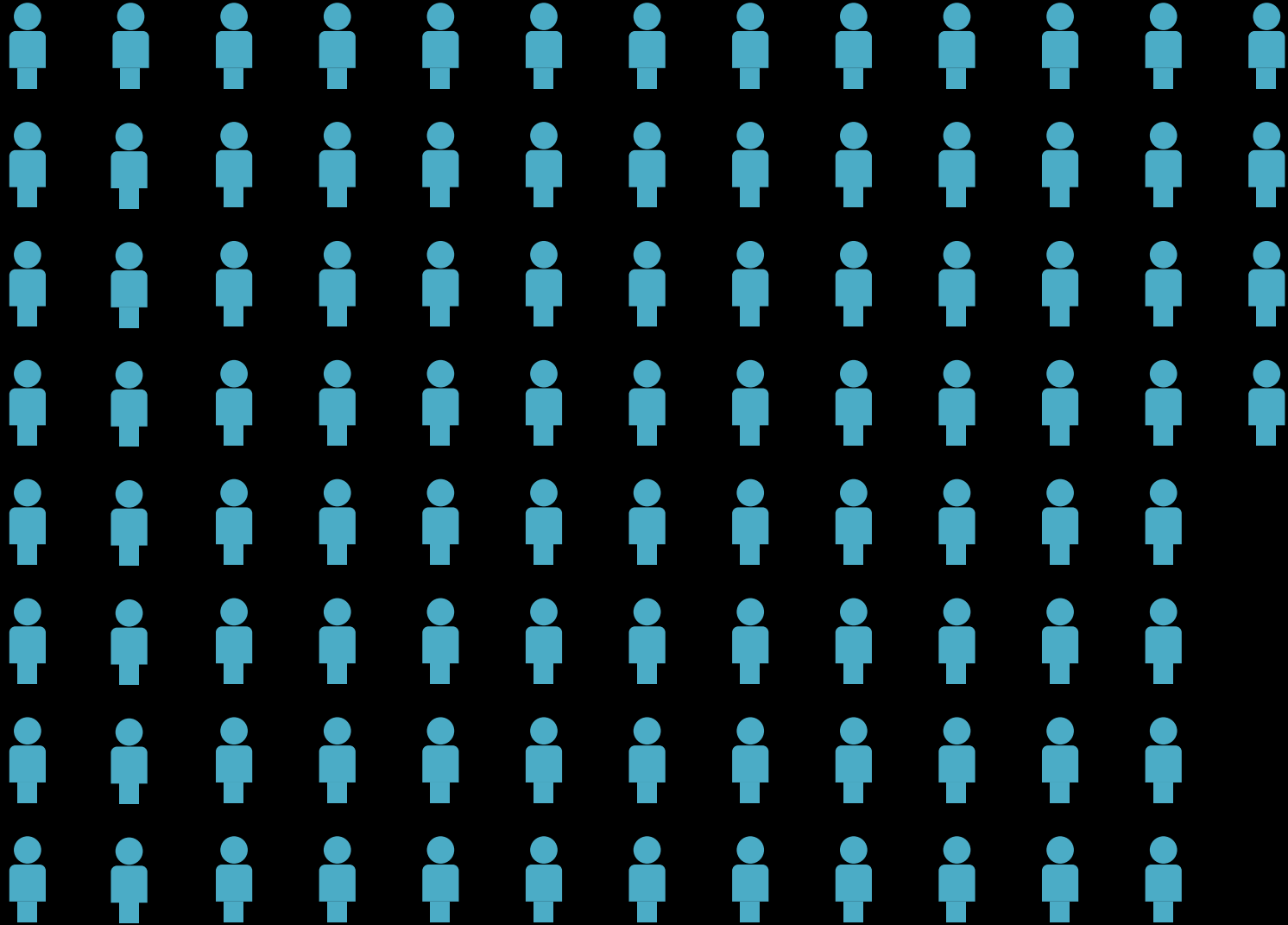
Learning to Program on the Internet



Task

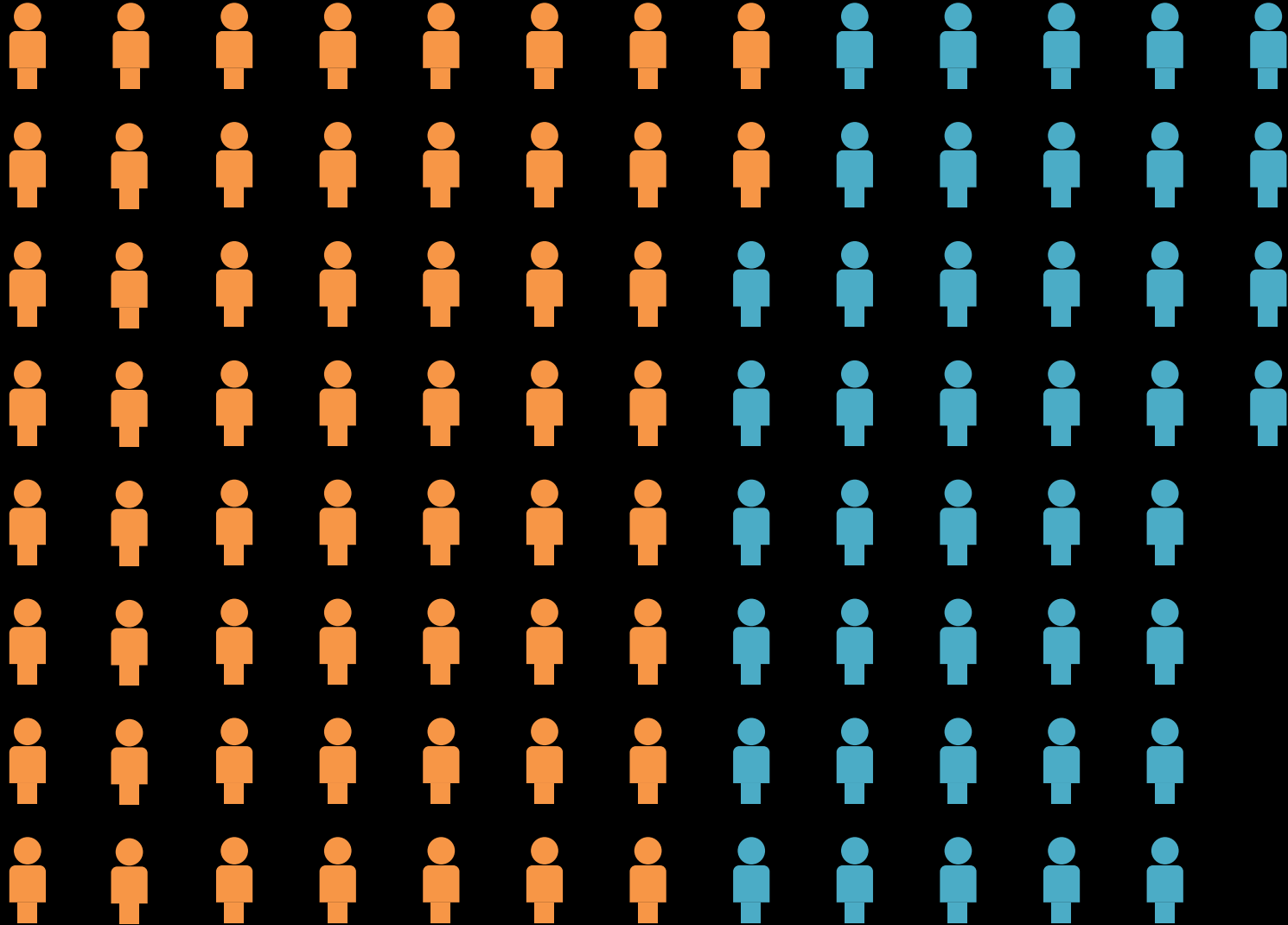
Almost a hundred thousand
unique solutions

US K-12 Students

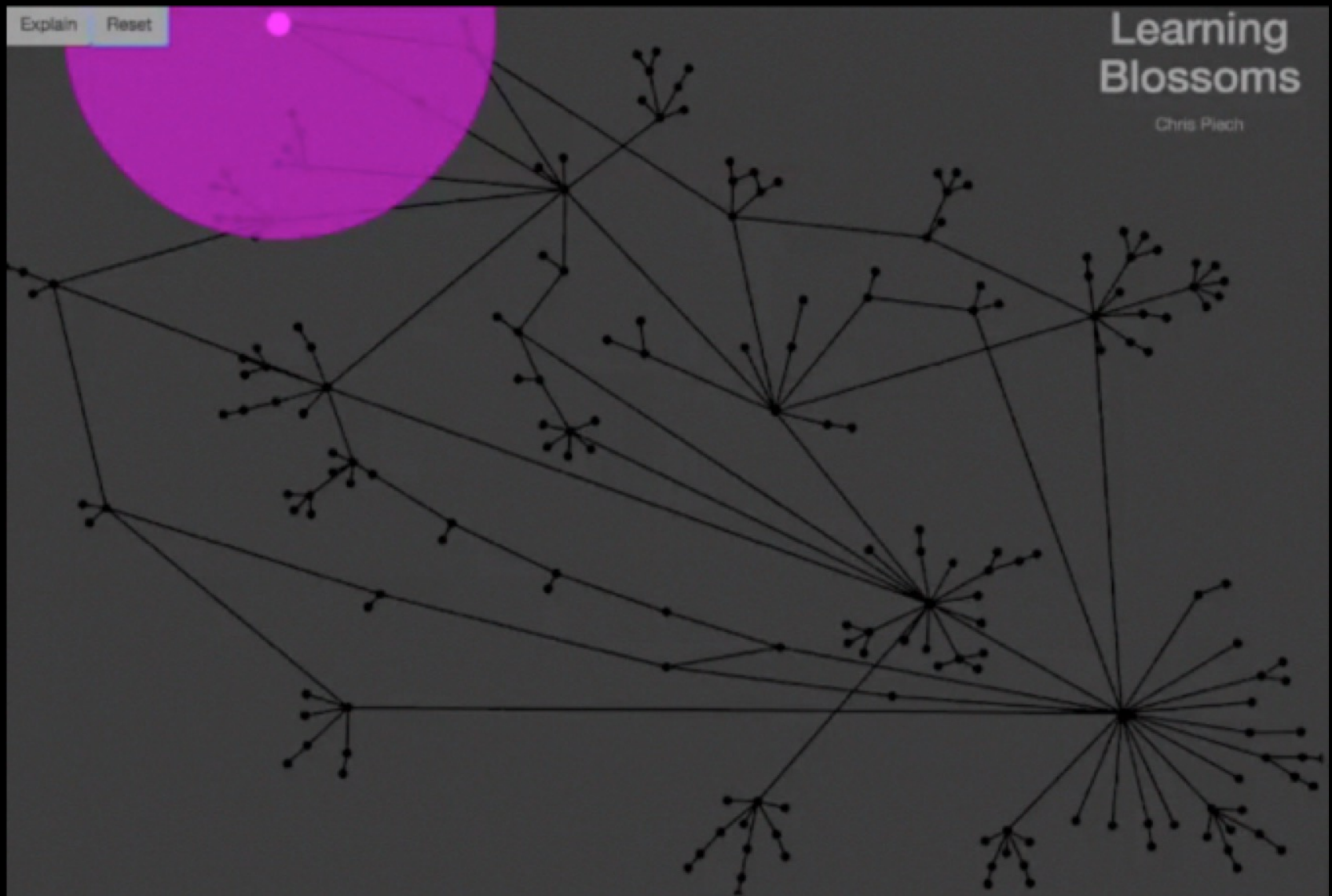


= 500,000 learners

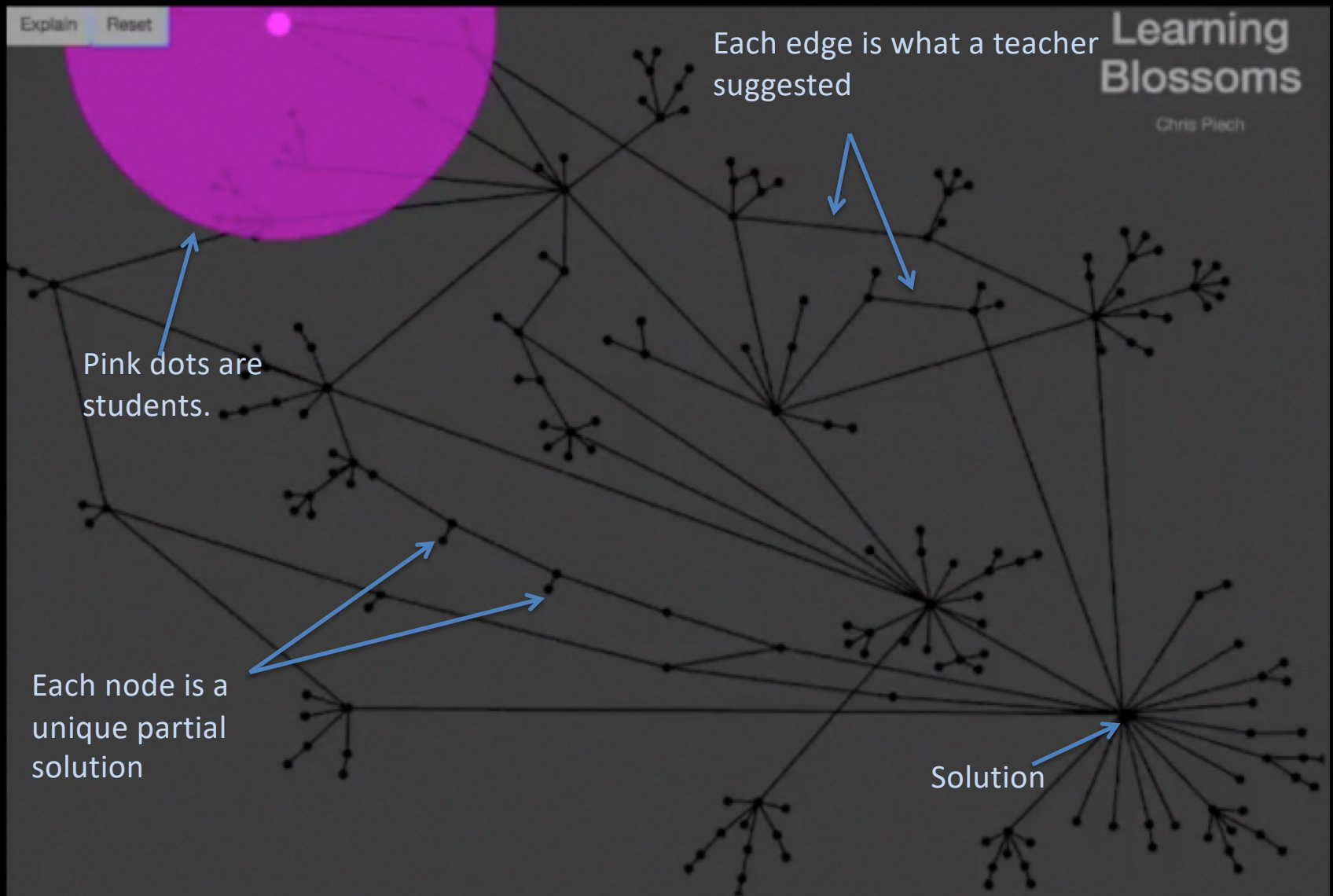
Code.org Students



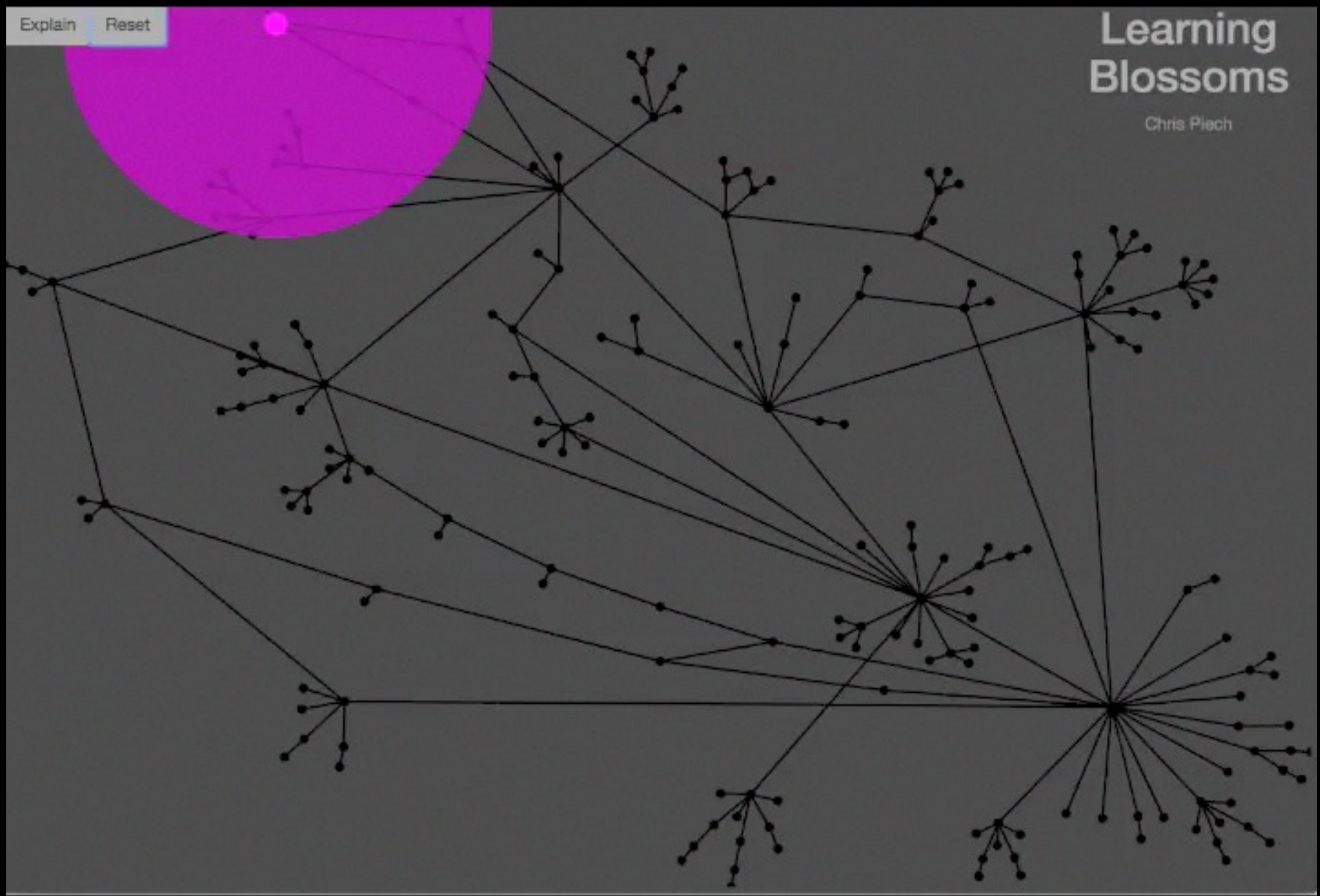
= 500,000 learners



Autonomously Generating Hints by Inferring Problem Solving Policies - Piech, Sahami et al.



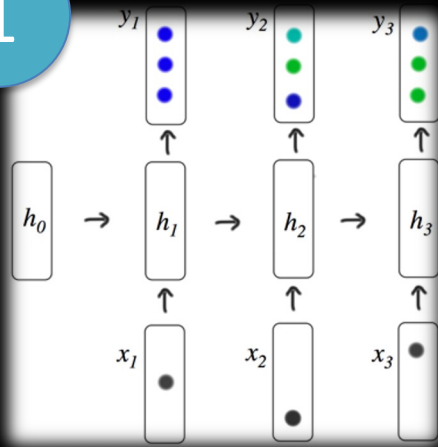
Autonomously Generating Hints by Inferring Problem Solving Policies - Piech, Sahami et al.



Autonomously Generating Hints by Inferring Problem Solving Policies - Piech, Sahami et al.

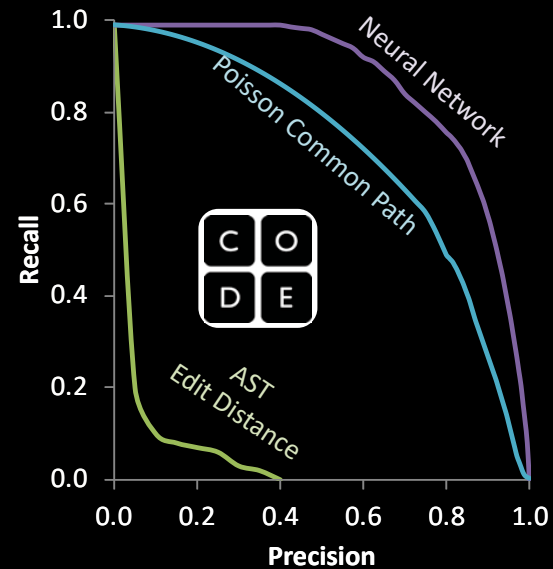
Deep Learning Algorithms

1

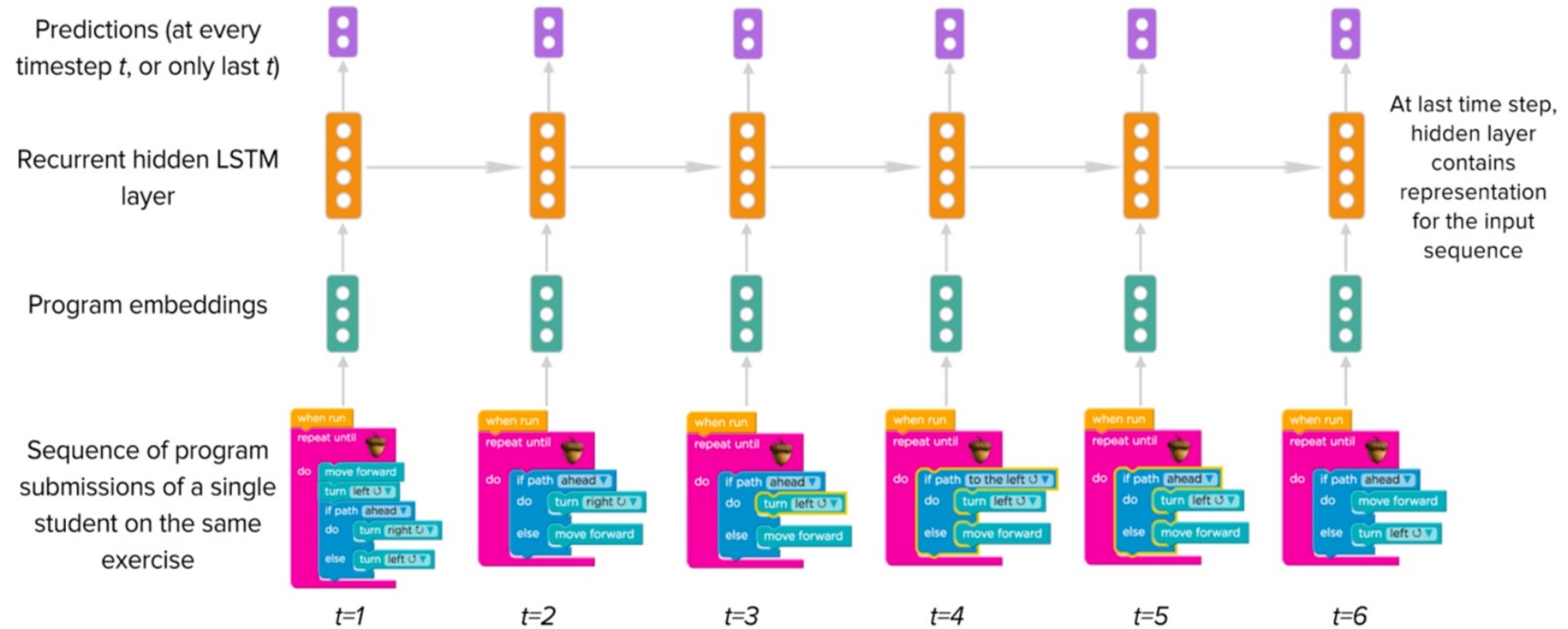


2

Program $\rightarrow \mathbb{R}^n$

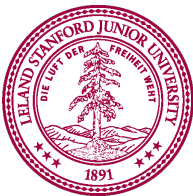


Deep Learning on Trajectories

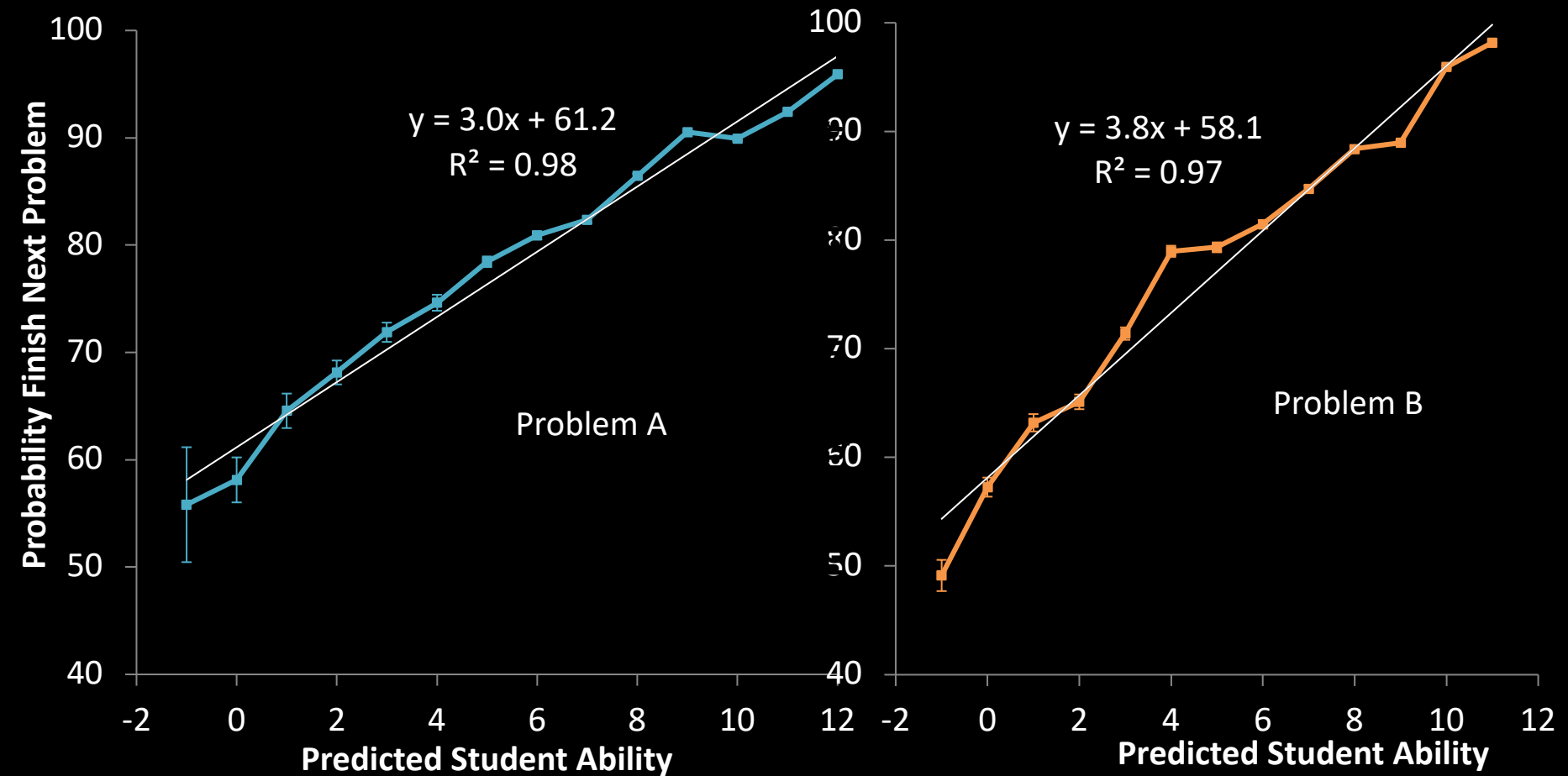


Research in collaboration with Lisa Wang

Piech + Sahami, CS106A, Stanford University



Predicts Future Success



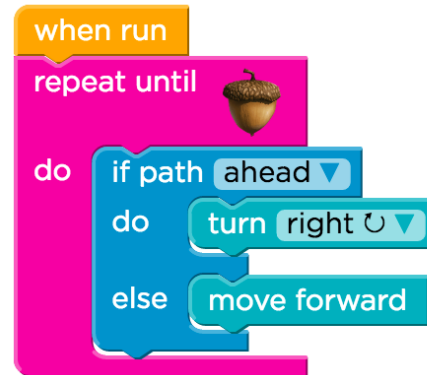
Most likely to succeed?

Highly Rates Grit

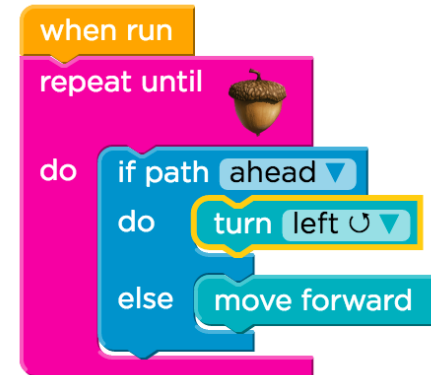
1. Two compound errors



2. Solves first error



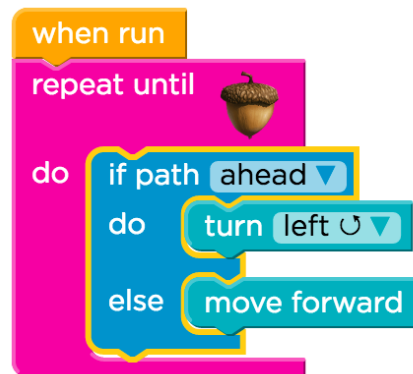
3. Starts reasonable attempt



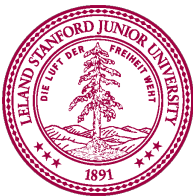
4. Completes attempt



5. Backtracks



6. Finds solution



Highly Rates Grit

1. Two compound errors



2. Solves first error



3. Starts reasonable attempt



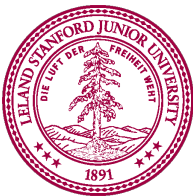
4. Completes attempt



5. Backtracks



6. Finds solution

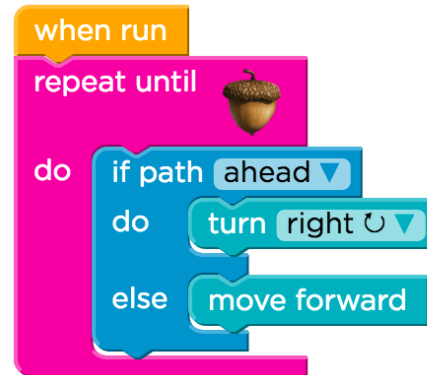


Highly Rates Grit

1. Two compound errors



2. Solves first error



3. Starts reasonable attempt



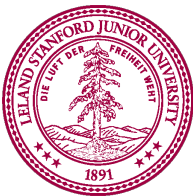
4. Completes attempt



5. Backtracks

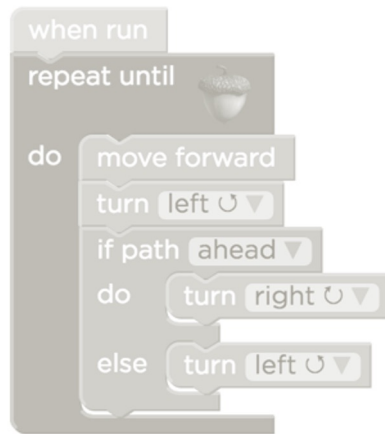


6. Finds solution



Highly Rates Grit

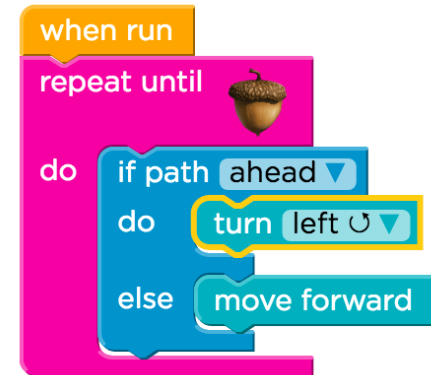
1. Two compound errors



2. Solves first error



3. Starts reasonable attempt



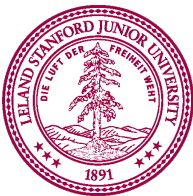
4. Completes attempt



5. Backtracks

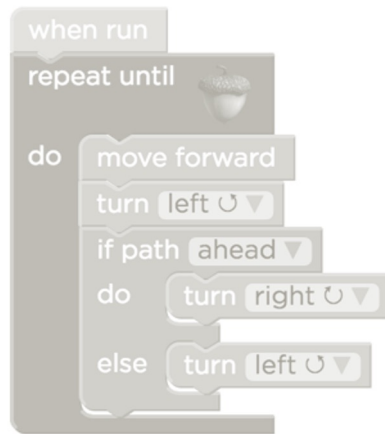


6. Finds solution



Highly Rates Grit

1. Two compound errors



2. Solves first error



3. Starts reasonable attempt



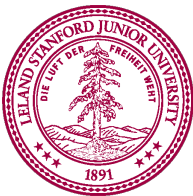
4. Completes attempt



5. Backtracks



6. Finds solution



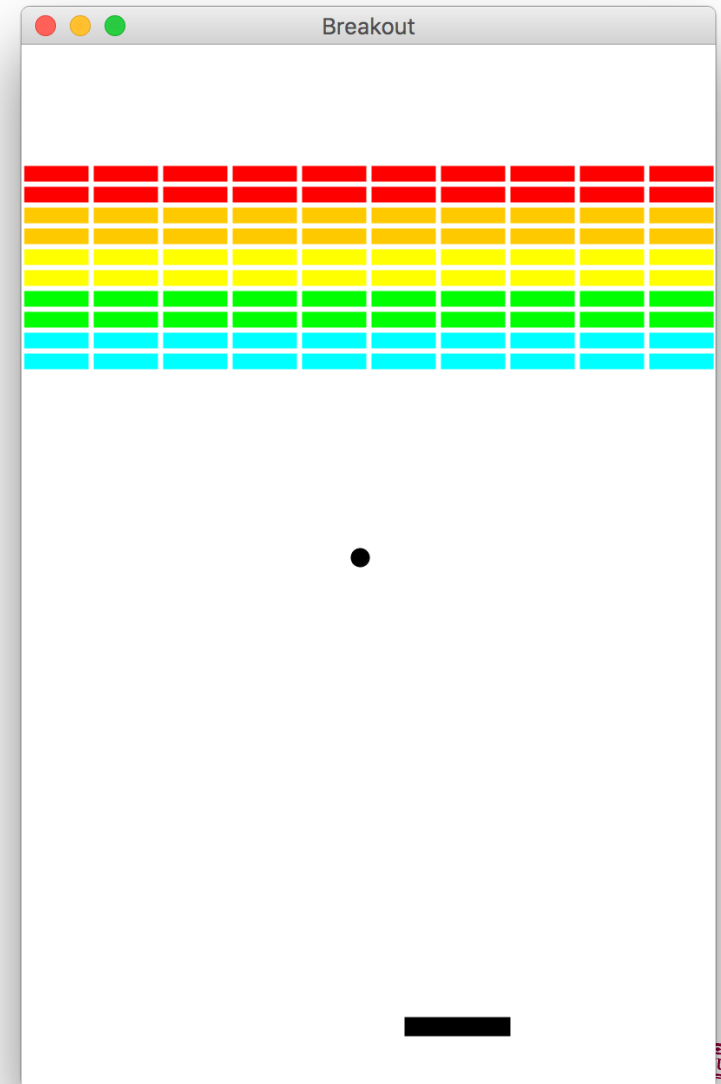
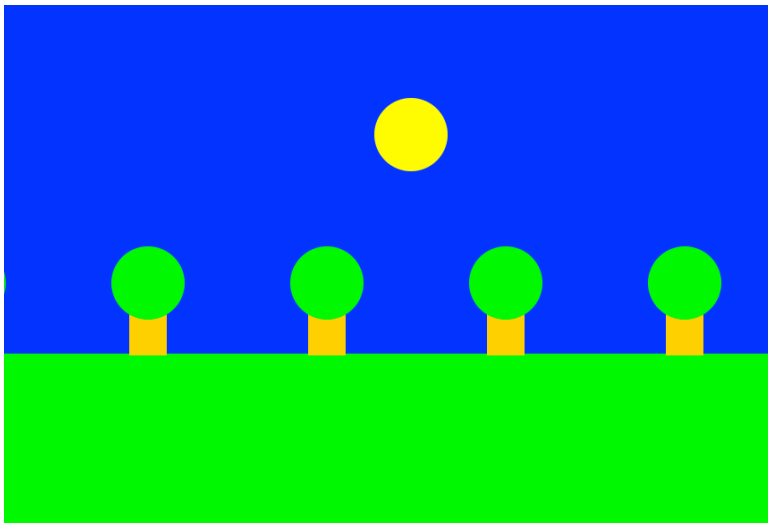
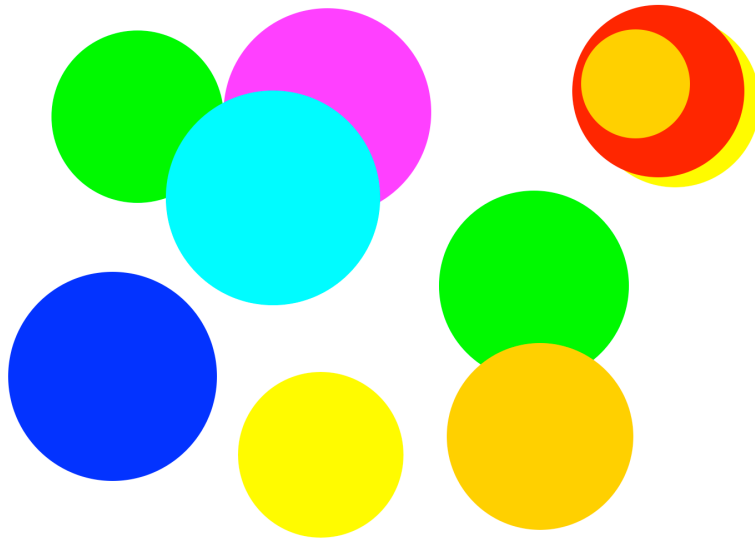
Coding is a new way to think
Keep challenging yourself.

Today's Goal

1. How do I draw things?



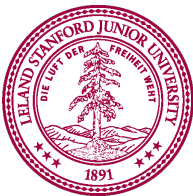
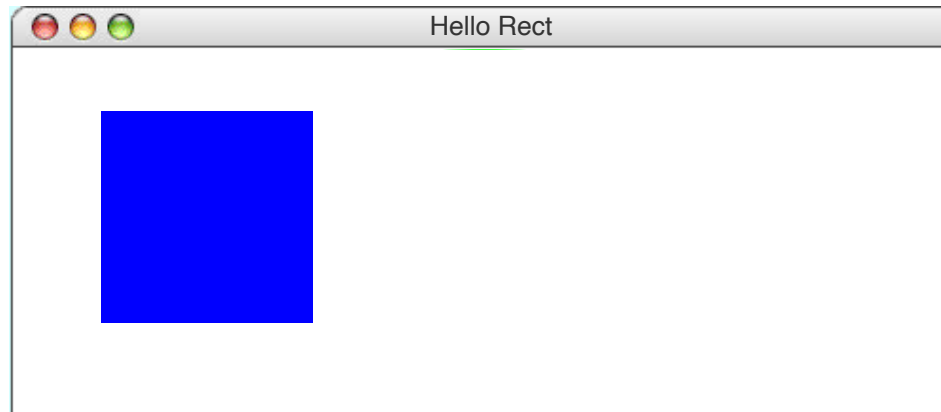
Graphics Programs



Draw a Rectangle

the following `main` method displays a blue square

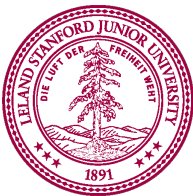
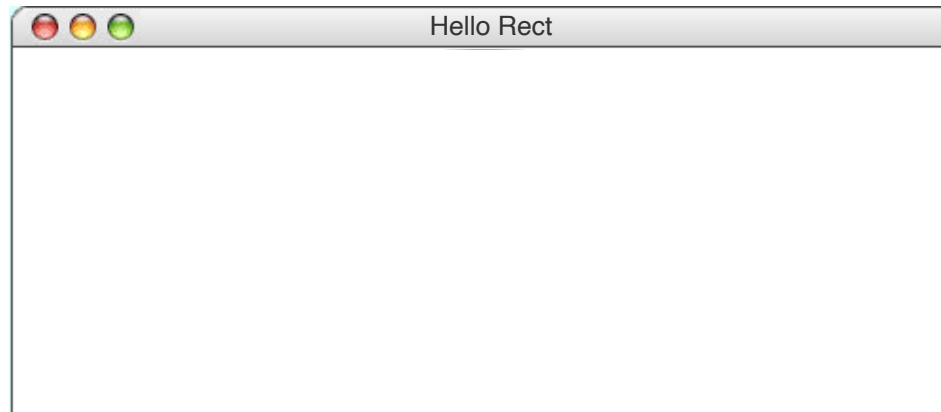
```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100, "blue")
```



Draw a Rectangle

the following `main` method displays a blue square

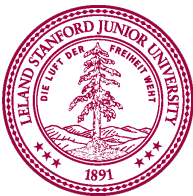
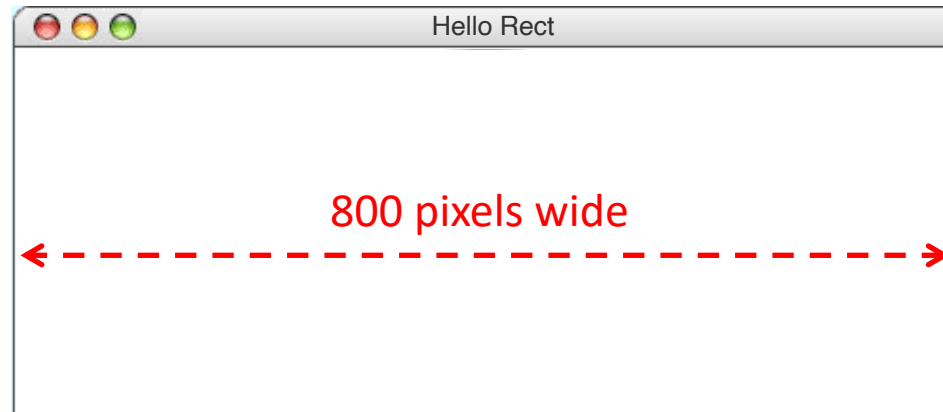
```
def main():  
    canvas = Canvas(800, 200)
```



Draw a Rectangle

the following `main` method displays a blue square

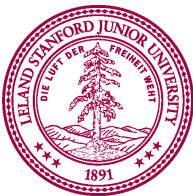
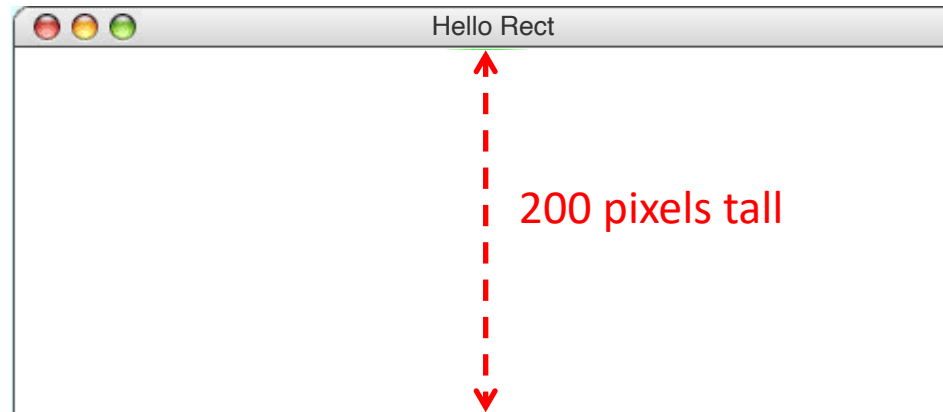
```
def main():  
    canvas = Canvas(800, 200)
```



Draw a Rectangle

the following `main` method displays a blue square

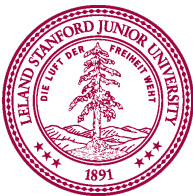
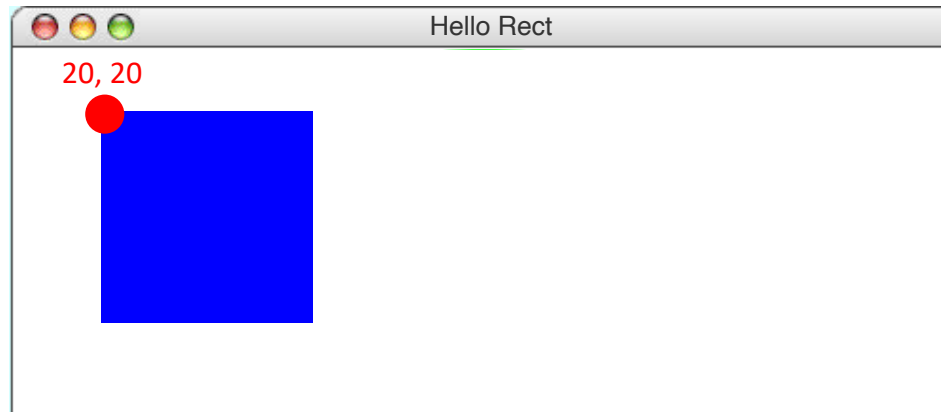
```
def main():  
    canvas = Canvas(800, 200)
```



Draw a Rectangle

the following `main` method displays a blue square

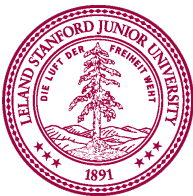
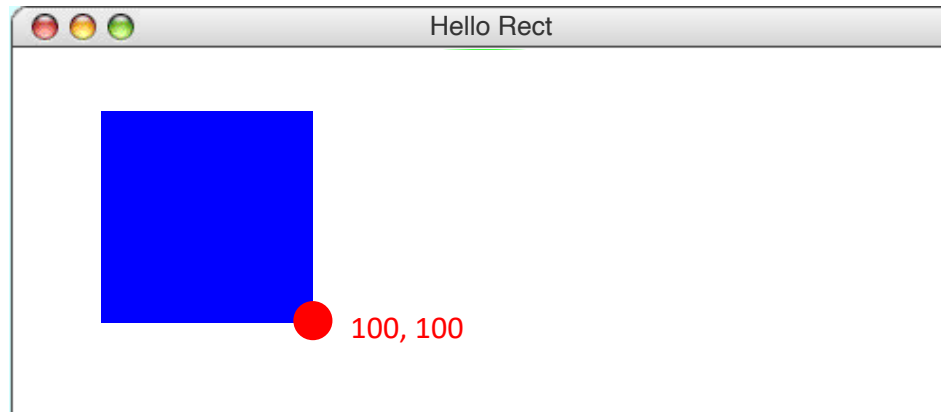
```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100, "blue")
```



Draw a Rectangle

the following `main` method displays a blue square

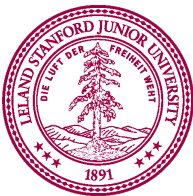
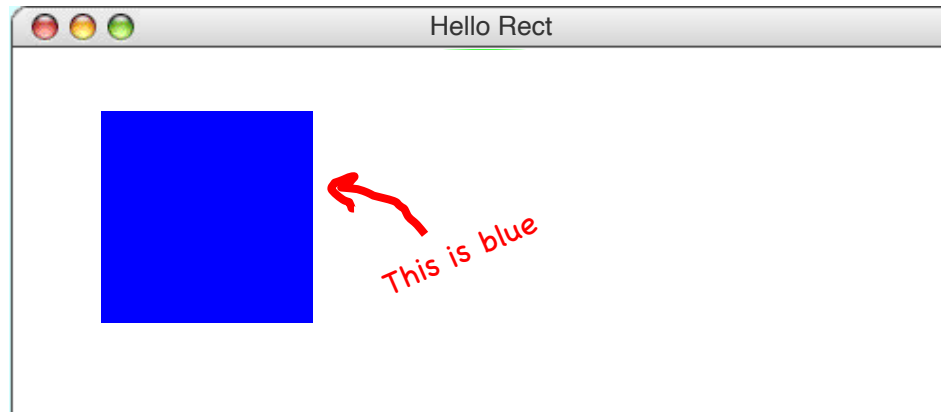
```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100, "blue")
```



Draw a Rectangle

the following `main` method displays a blue square

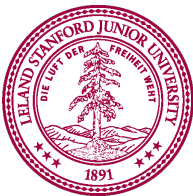
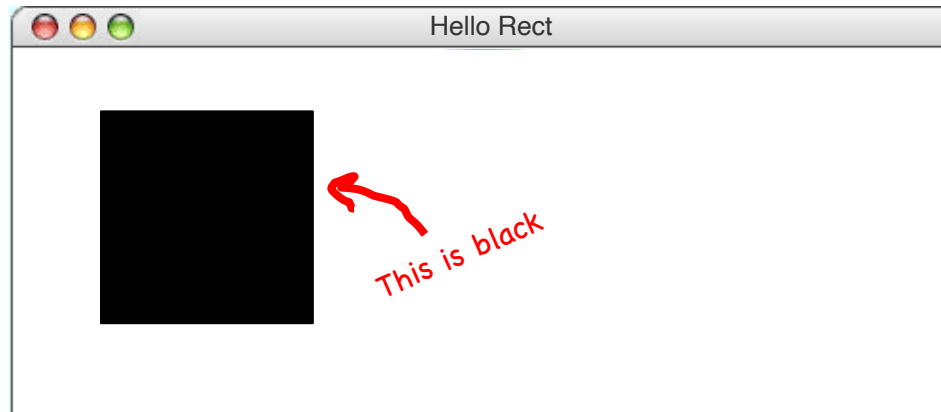
```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100, "blue")
```



Draw a Rectangle

the following `main` method displays a blue square

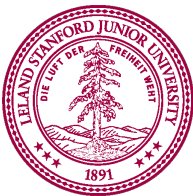
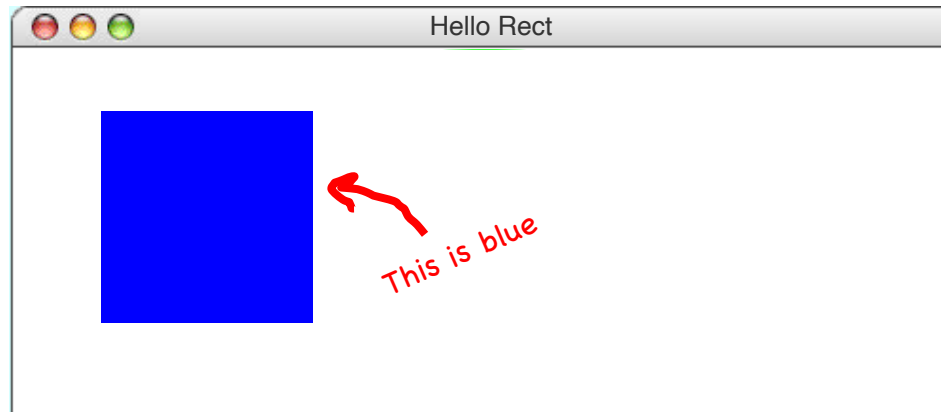
```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100)
```



Draw a Rectangle

the following `main` method displays a blue square

```
def main():  
    canvas = Canvas(800, 200)  
    canvas.create_rectangle(20, 20, 100, 100, "blue")
```



TK Natural Graphics



Graphics Coordinates

0,0

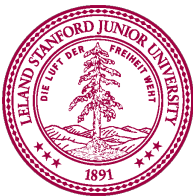
x 40,20

x 120,40

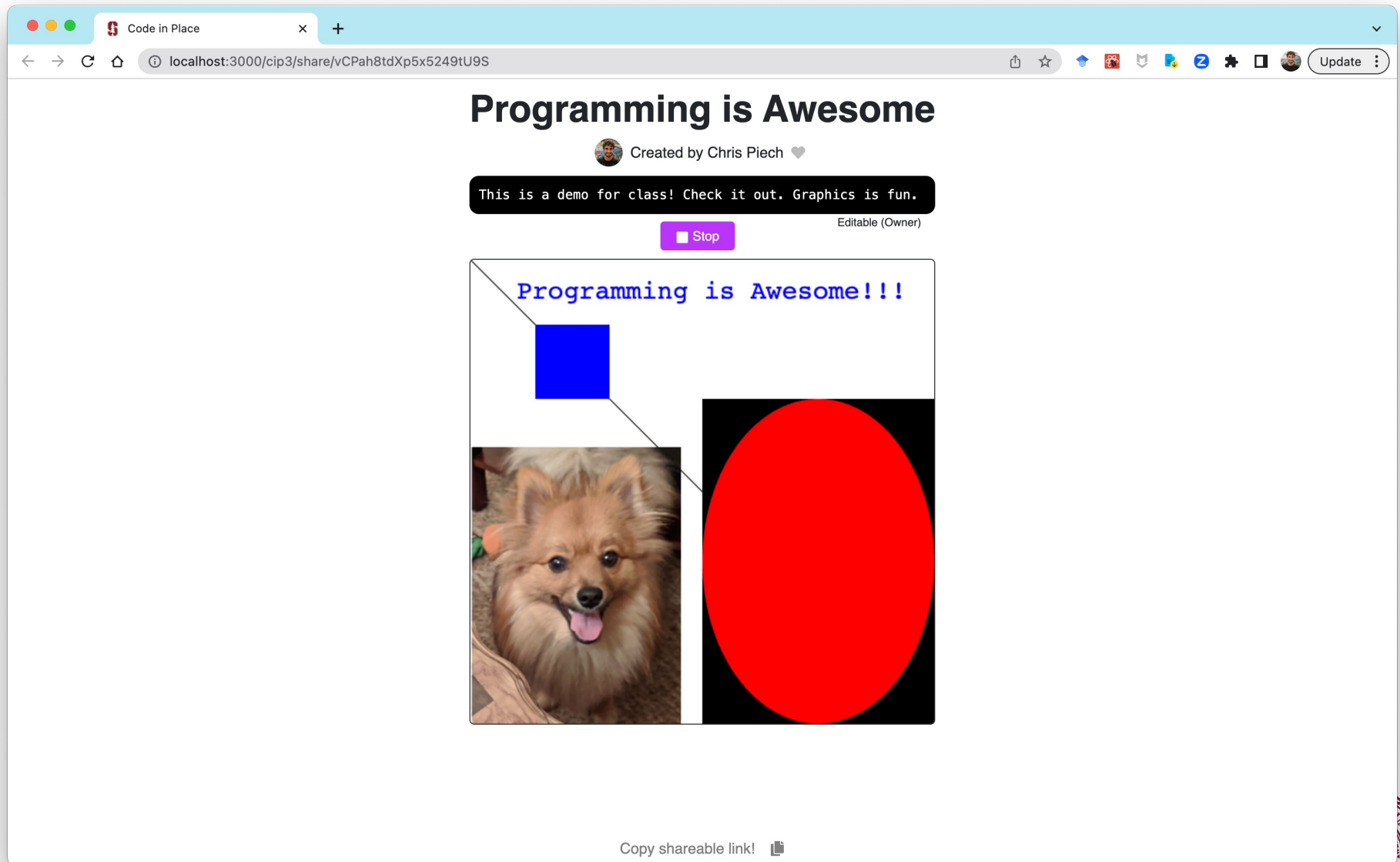
x 40,120

CANVAS_WIDTH

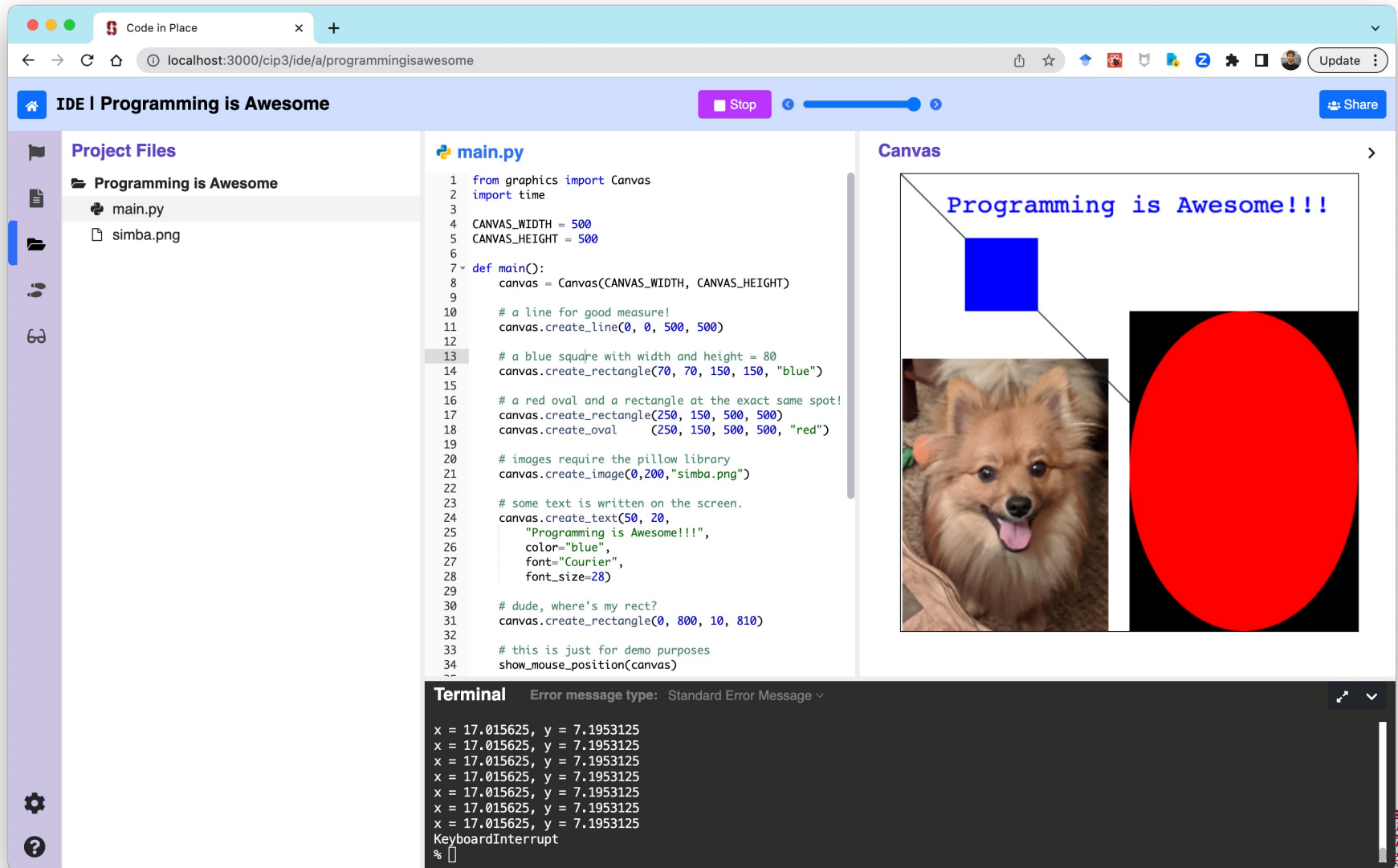
CANVAS_HEIGHT



Rectangles, Ovals, Text



Rectangles, Ovals, Text



- `canvas.create_line()`
- `canvas.create_oval()`
- `canvas.create_text()`



- `canvas.create_line(x1, y1, x2, y2)`
- `canvas.create_oval()`
- `canvas.create_text()`



- `canvas.create_line(x1, y1, x2, y2)`

- `canvas.create_oval()`

- `canvas.create_text()`

The first point of the
line is (x1, y1)



- `canvas.create_line(x1, y1, x2, y2)`

- `canvas.create_oval()`

- `canvas.create_text()`



The second point of the
line is `(x2, y2)`



- `canvas.create_line(x1, y1, x2, y2)`

- `canvas.create_oval()`

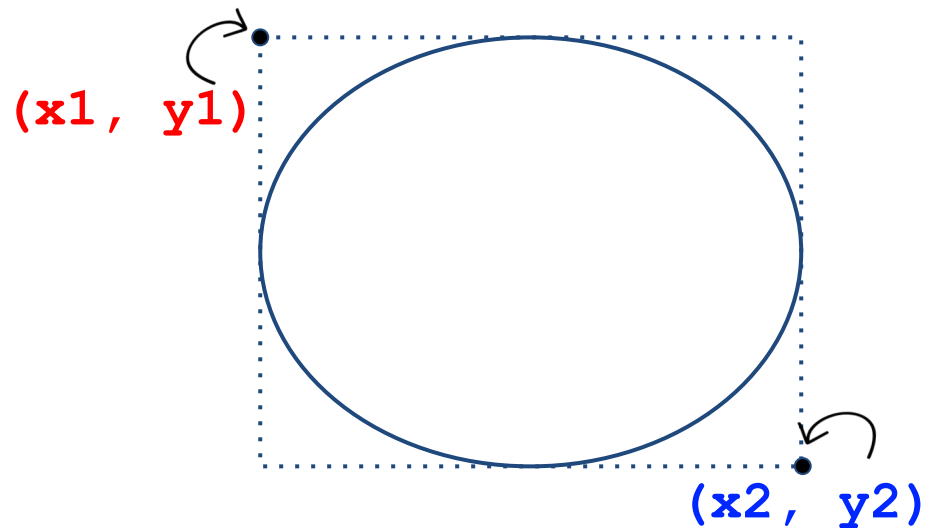
- `canvas.create_text()`



- `canvas.create_line()`
- `canvas.create_oval(x1, y1, x2, y2)`
- `canvas.create_text()`



- `canvas.create_line()`
- `canvas.create_oval(x1, y1, x2, y2)`
- `canvas.create_text()`

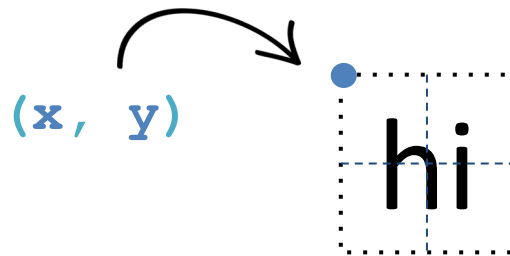


- `canvas.create_line()`
- `canvas.create_oval()`
- `canvas.create_text(x, y, text='hi')`

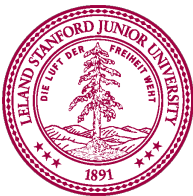
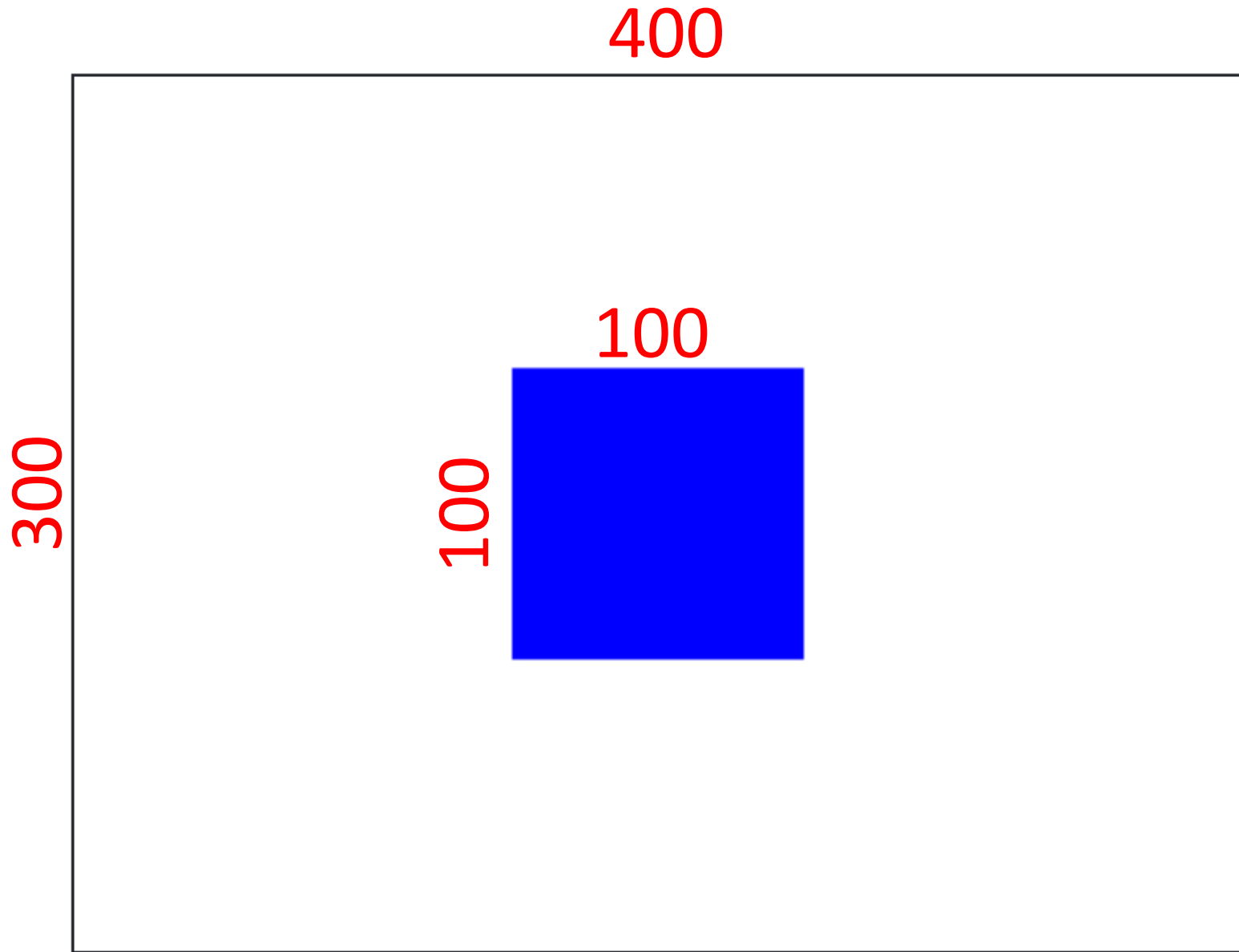
hi



- `canvas.create_line()`
- `canvas.create_oval()`
- `canvas.create_text(x, y, text='hi')`



Centered Square



Centered Square

The screenshot shows the Code in Place IDE interface. The browser address bar indicates the URL is `localhost:3000/cip3/ide/a/centeredsquare`. The IDE title is "IDE | Centered Square".

Instructions:

Write a program that draws a square, 200 pixels wide, by 200 pixels high, right in the center of the canvas. Warning: this example involves a little math. But it is math worth knowing!

This program is hard, because the first attempt that most students try does not work. It is tempting to try to place the square in the middle by setting the first two parameters of `create_rectangle` to be `middle_x` and `middle_y`:

```
middle_x = CANVAS_WIDTH/2
middle_y = CANVAS_HEIGHT/2

# calculate the right and bottom of the square
right_x = middle_x + SQUARE_SIZE
bottom_y = middle_y + SQUARE_SIZE

# draw the square
canvas.create_rectangle(middle_x,
middle_y, right_x, bottom_y, 'blue')
```

However, instead of putting the square in the middle, this puts the top left corner of the square in the middle of the canvas.

main.py

```
1 from graphics import Canvas
2
3 CANVAS_WIDTH = 400
4 CANVAS_HEIGHT = 300
5 SQUARE_SIZE = 100
6
7 def main():
8     canvas = Canvas(CANVAS_WIDTH, CANVAS_HEIGHT)
9
10    # get the middle of the canvas
11    middle_x = CANVAS_WIDTH/2
12    middle_y = CANVAS_HEIGHT/2
13
14    # calculate the top left of the square
15    left_x = middle_x
16    top_y = middle_y
17
18    # calculate the right and bottom of the square
19    right_x = left_x + SQUARE_SIZE
20    bottom_y = top_y + SQUARE_SIZE
21
22    # draw the square
23    canvas.create_rectangle(left_x, top_y, right_x, bottom_y, 'blue')
24    print("Done!")
25
26 if __name__ == '__main__':
27     main()
```

Canvas

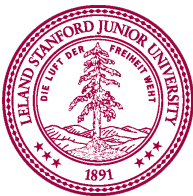
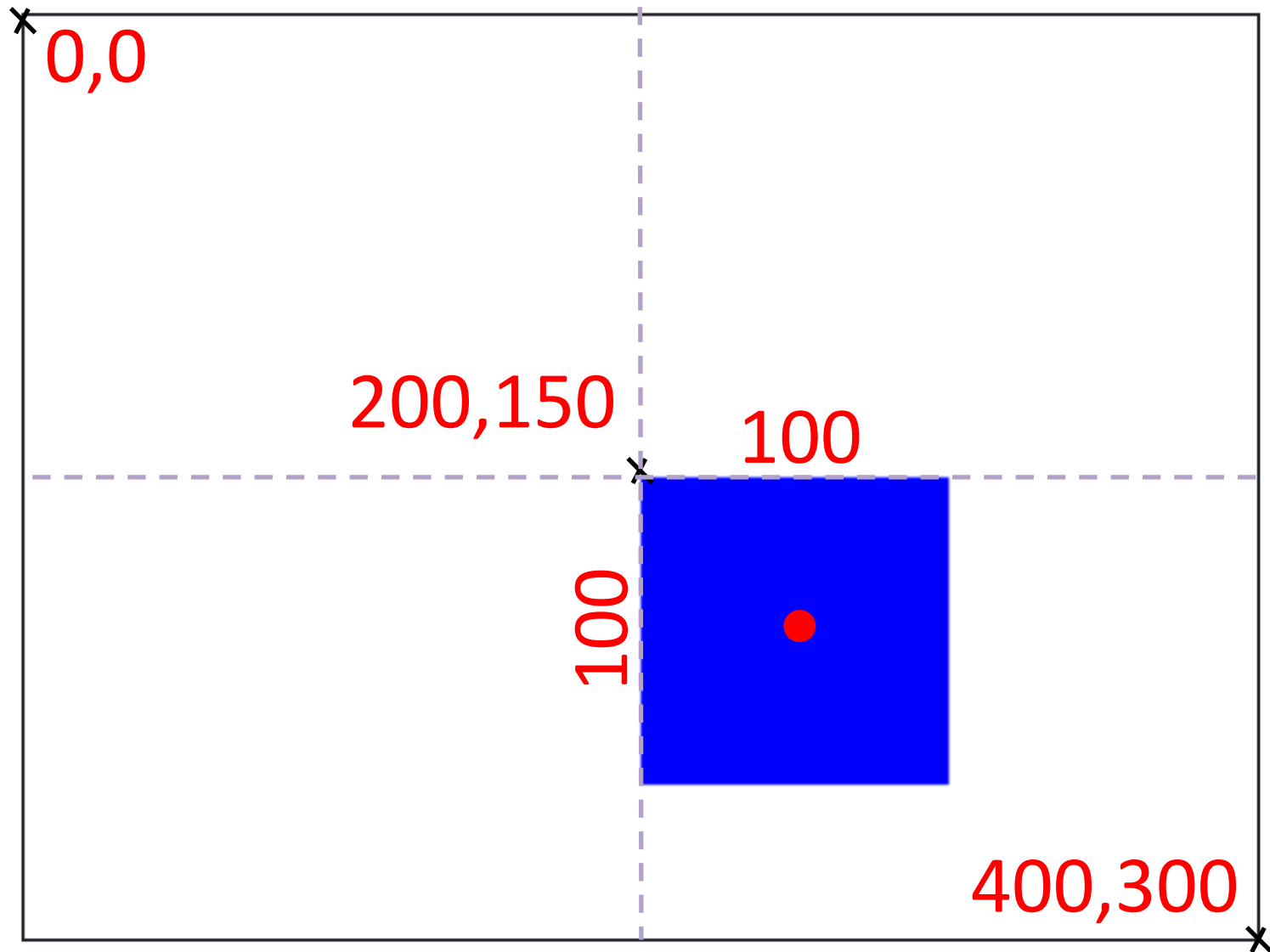
The canvas shows a blue square centered within a white rectangular frame.

Terminal

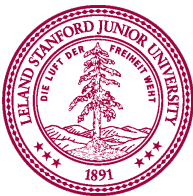
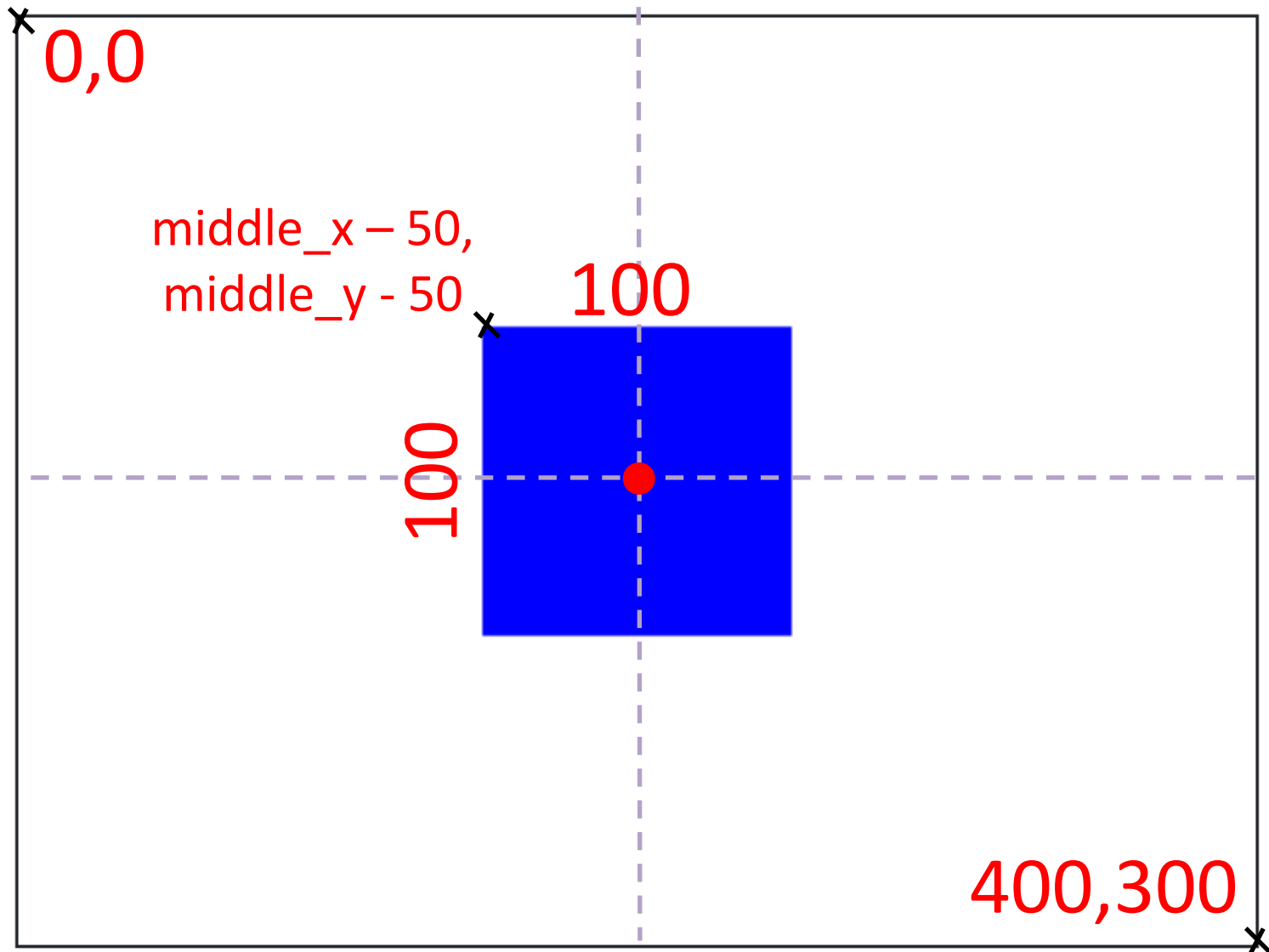
```
Error message type: Standard Error Message
Done!
% python main.py
Done!
% python main.py
Done!
%
% python main.py
Done!
% 
```



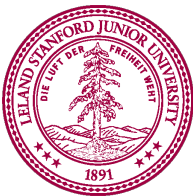
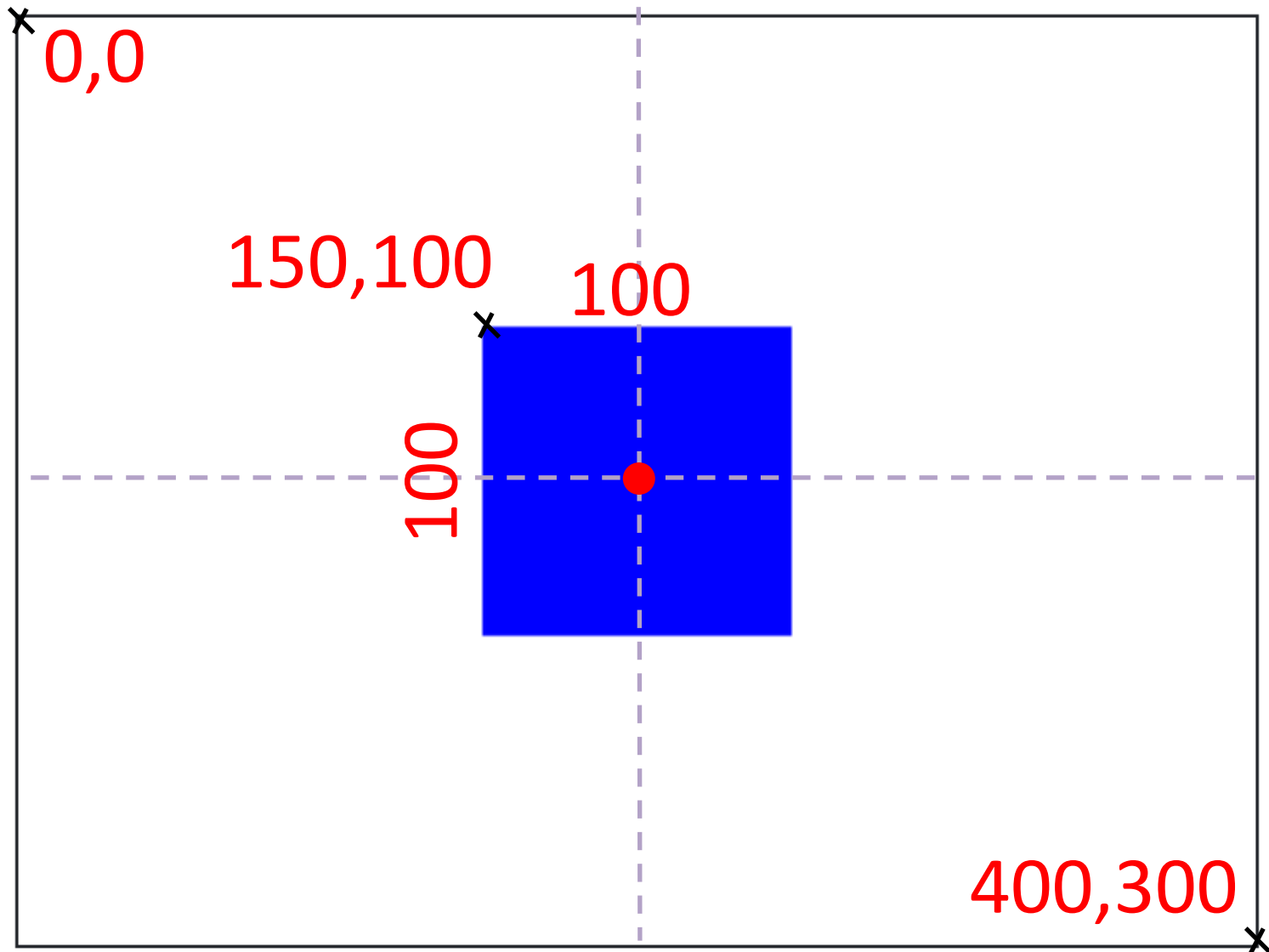
Centered Square



Centered Square



Centered Square



Centered Square

IDE | Centered Square [Run] [Share]

Instructions [Restart]

Write a program that draws a square, 200 pixels wide, by 200 pixels high, right in the center of the canvas. Warning: this example involves a little math. But it is math worth knowing!

This program is hard, because the first attempt that most students try does not work. It is tempting to try to place the square in the middle by setting the first two parameters of `create_rectangle` to be `middle_x` and `middle_y`:

```
middle_x = CANVAS_WIDTH/2
middle_y = CANVAS_HEIGHT/2

# calculate the right and bottom of the square
right_x = middle_x + SQUARE_SIZE
bottom_y = middle_y + SQUARE_SIZE

# draw the square
canvas.create_rectangle(middle_x,
middle_y, right_x, bottom_y, 'blue')
```

However, instead of putting the square in the middle, this puts the top left corner of the square in the middle of the canvas.

main.py

```
1 from graphics import Canvas
2
3 CANVAS_WIDTH = 400
4 CANVAS_HEIGHT = 300
5 SQUARE_SIZE = 100
6
7 def main():
8     canvas = Canvas(CANVAS_WIDTH, CANVAS_HEIGHT)
9
10    # get the middle of the canvas
11    middle_x = CANVAS_WIDTH/2
12    middle_y = CANVAS_HEIGHT/2
13
14    # calculate the top left corner position
15    left_x = middle_x - SQUARE_SIZE/2
16    top_y = middle_y - SQUARE_SIZE/2
17
18    # calculate the right and bottom of the square
19    right_x = left_x + SQUARE_SIZE
20    bottom_y = top_y + SQUARE_SIZE
21
22    # draw the square
23    canvas.create_rectangle(left_x, top_y, right_x, bottom_y, 'blue')
24
25 if __name__ == '__main__':
26     main()
```

Canvas

Terminal Error message type: Standard Error Message

```
% python main.py
% python main.py
% python main.py
% python main.py
% python main.py
% python main.py
%
```



Step by Step

Code in Place x Code in Place x Code in Place +

localhost:3000/cip3/ide/a/rectline

IDE | Rect Line Replay

Run

Share

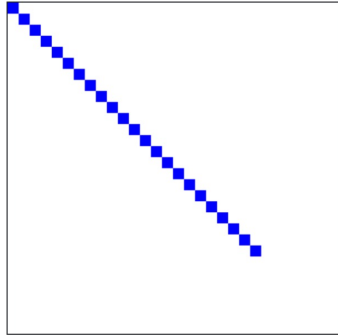
Replay Mode

Variable	Value
canvas	<Canvas>
i	23
left_x	220
top_y	220
right_x	230
bottom_y	230

main.py

```
1 import graphics
2
3 def main():
4     # makes a canvas
5     canvas = graphics.create_canvas(300, 300)
6     # make 20 some blue squares
7     for i in range(30):
8         # it can be helpful to store each coord into its own variable
9         left_x = i * 10
10        top_y = i * 10
11        right_x = left_x + 10
12        bottom_y = top_y + 10
13        canvas.create_rectangle(left_x, top_y, right_x, bottom_y, 'blue')
14
15        # keep track of i
16        print(i)
17
18 if __name__ == '__main__':
19     main()
```

Replay Canvas



Replay Terminal

```
13
14
15
16
17
18
19
20
21
22
```



Pedagogy

Conditionals

Loops

Nested Loops

Random

Nonetypes

Advanced Arithmetic

Floating Point

Graphics

Basic Shapes

Animation

Functions

Anatomy of a Function

Scope

Parameters

Return vs Print

Multiple Returns

Function Decomposition

Update Functions

Code in Place

Code in Place

codeinplace.stanford.edu/cip3/textbook/shapes

Update

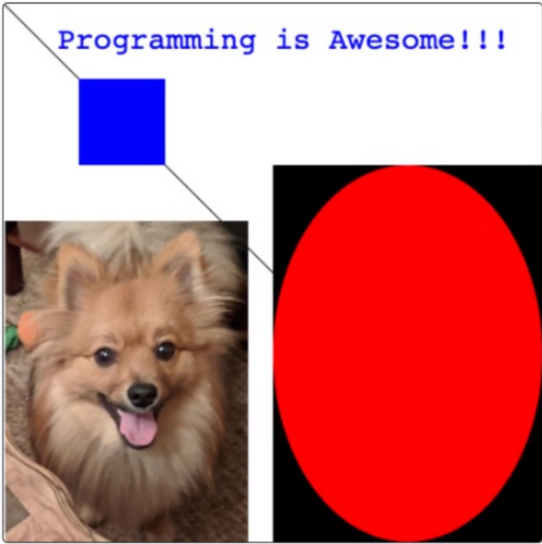
Basic Shapes

Introduction to Graphics

Adapted from the Stanford CS106A Graphics Reference

Quest

Graphics!!!! Yep, it's time to give Python an upgrade! All the apps on your computer, everything from the calculator to the file explorer to the very browser you might be reading this in right now, are all programs that use graphics to make interactive, visual programs. You're about to get your first taste of graphics by learning how to draw shapes on the screen. Ready? By the end of this chapter you should be able to make programs that can draw!



Text vs Graphics